```
    static void increase_external_count(
         std::atomic<counted_node_ptr>& counter,
         counted_node_ptr& old_counter)
    {
         counted_node_ptr new_counter;

        do
        {
            new_counter=old_counter;
            ++new_counter.external_count;
        }
        while(!counter.compare_exchange_strong(
               old_counter,new_counter,

std::memory_order_acquire,std::memory_order_relaxed));

        old_counter.external_count=new_counter.external_count;
    }
};
```

Listing 7.18 is the other side. This time, rather than releasing a reference, you're obtaining a fresh one and increasing the external count. `increase_external_count()` is similar to the `increase_head_count()` function from listing 7.12, except that it has been made into a `static` member function that takes the external counter to update as the first parameter rather than operating on a fixed counter.

Listing 7.19. Freeing an external counter to a node in a lock-free queue